EIE330 FPGA-based System Design

Lab 1 完整操作指南

全加器 | 一位数字比较器

本指南包含：

1. 两个任务从需求到上板的完整六步流程；
2. 可直接使用的 Verilog 设计代码与 Testbench；
3. iverilog / yosys / GTKWave / 波形绘图的全部命令；
4. 开发板测试、引脚映射与视频录制方法；
5. 实验报告 a–h 各部分写作模板与检查清单。

姓名 / 学号：_____

小组成员：_____

目 录

第 1 章 实验总览：要做什么、交什么 4

1.1 两个任务 4

1.2 每个任务都要走完的六步流程 4

1.3 最终要提交的东西 4

第 2 章 准备工作 5

2.1 需要安装的软件 5

2.2 确认工具安装成功 5

2.3 建立工程目录与 GitHub 仓库 5

第 3 章 六步开发流程（总览） 7

第 4 章 任务 1-1：全加器（手把手） 9

4.1 Step 1 调研：需求与真值表 9

4.2 Step 2 设计：表达式、框图与预期波形 9

4.3 Step 3 编写 RTL 设计代码 10

4.4 Step 3 语法检查与逻辑综合 11

4.5 Step 4 编写 Testbench 并功能仿真 12

4.6 Step 5 时序分析与波形绘图 13

4.7 Step 6 上板测试 15

第 5 章 任务 1-2：一位数字比较器（手把手） 16

5.1 Step 1 调研：需求与真值表 16

5.2 Step 2 设计 16

5.3 Step 3 RTL 代码与综合 17

5.4 Step 4 Testbench 与仿真 18

5.5 Step 5 时序分析与波形绘图 19

5.6 Step 6 上板测试 19

第 6 章 开发板测试通用流程 21

6.1 引脚约束文件 21

6.2 综合、布局布线与烧录 21

6.3 逐行验证与视频录制 21

第 7 章 实验报告怎么写 23

7.1 封面 23
7.2 a) Introduction（引言） 23
7.3 b) Requirements analysis（需求分析） 23
7.4 c) System design（系统设计） 23
7.5 d) Implementation（实现） 23
7.6 e) Testing and results（测试与结果）—— 权重最高 23
7.7 f) Discussion（讨论，建议写） 24
7.8 g) Conclusion（结论） 24
7.9 h) References（参考文献） 24
7.10 图表与代码规范（硬性要求） 24
第 8 章 提交前总检查清单 25
8.1 流程与代码检查 25
8.2 报告检查 25
8.3 最终提交 25
附录 A 命令速查表 26
附录 B 常见报错与排查 26
附录 C 报告封面模板（文字） 26

第 1 章 实验总览：要做什么、交什么

1.1 两个任务

Lab 1 包含两个独立的小设计，两个都必须完成：

- 任务 1-1 全加器（Full Adder）：3 个按键作为输入 A、B、Cin（低位进位），2 个 LED 作为输出 S（和）、Cout（向高位进位），需要验证全部 8 种输入组合。
- 任务 1-2 一位数字比较器（Comparator）：2 个按键作为输入 A、B，3 个 LED 分别表示 A<B、A=B、A>B，需要验证全部 4 种输入组合。

用户看到的现象统一规定：按键按下 = 逻辑 1，松开 = 逻辑 0；LED 亮 = 输出 1，灭 = 0。

> 最关键的硬件约束：开发板上的按键和 LED 都是低电平有效（active-low）：按键按下时 FPGA 收到的是 0，LED 点亮需要 FPGA 输出 0。代码必须在端口处做取反转换，否则现象会完全相反。本指南所有示例代码已经处理好这一点。

1.2 每个任务都要走完的六步流程

两个任务都必须严格按课程讲授的流程顺序执行，每一个验证步骤失败都要返回修改后重走：

- Step 1 前期调研：理清需求、器件约束（低有效、FPGA 资源）、查阅资料；
- Step 2 方案设计：写真值表、画框图、画预期波形、确定设计决策（写代码之前完成）；
- Step 3 编写代码：RTL 与 testbench 分目录存放，用 iverilog 查语法、yosys 综合；
- Step 4 功能仿真：testbench 施加全部输入组合，生成 .vcd 波形，用 GTKWave 调试；
- Step 5 时序分析：把仿真波形与预期波形逐行对比，并用仿真数据绘制报告用波形图；
- Step 6 上板测试：规划引脚映射，逐行验证真值表，录制验证视频。

1.3 最终要提交的东西

- 一个 PDF 文件：两个任务的报告合并而成；每个任务的报告有独立封面（任务名、姓名、学号）；
- 一个 GitHub 仓库：存放全部 Verilog 代码、testbench、综合脚本，报告中给出链接；
- 板级验证视频：上传到视频网站，报告中给出链接（截图只能作为补充材料）；
- 报告中按 IEEE 格式列出参考文献；如使用 AI 辅助，必须按课程 AI 政策声明使用部分。

> 评分看重什么：分数不只看电路能不能跑，更看重你是否完整、诚实地展示了开发流程：需求是否可追溯、设计决策是否有理由、仿真是否覆盖全部情况、波形是否由数据绘制、上板是否逐行验证、是否理解每个工具的局限。代码或报告抄袭一律零分。

第 2 章 准备工作

2.1 需要安装的软件

本实验用到 4 类工具：

- iverilog（Icarus Verilog）：Verilog 编译器，做语法检查与功能仿真；
- vvp：iverilog 自带的仿真运行命令；
- GTKWave：查看 .vcd 波形的图形工具（调试用）；
- yosys：逻辑综合工具，生成 RTL 视图、查看资源占用；show 命令还需要 graphviz；
- Python 3 + vcdvcd / pandas / matplotlib：把 VCD 转 CSV、绘制报告用波形图；
- 以及课程开发板对应的下载/烧录工具链（以课程指定的开发板为准）。

Ubuntu / Debian 下一次性安装：

```
sudo apt update
sudo apt install -y iverilog gtkwave yosys graphviz python3-pip python3-tk
pip3 install --user vcdvcd pandas matplotlib
```

macOS（Homebrew）：

```
brew install icarus-verilog gtkwave yosys graphviz
pip3 install vcdvcd pandas matplotlib
```

Windows 推荐使用课程提供的 Linux 虚拟机 / WSL，在 WSL（Ubuntu）中执行上面的 apt 命令。

2.2 确认工具安装成功

```
iverilog -V        # 能打印版本号即安装成功
yosys -V
gtkwave --version
python3 -c "import vcdvcd, pandas, matplotlib; print('python packages OK')"
```

2.3 建立工程目录与 GitHub 仓库

建议为 Lab 1 建立一个 Git 仓库，两个任务各占一个文件夹，设计代码放 RTL/，测试代码放 sim/（这是要求明确规定的）：

```
eie330-lab1/
├── full_adder/
│   ├── RTL/
│   │   └── full_adder.v        # 设计代码
│   ├── sim/
│   │   └── tb_full_adder.v     # 测试平台
│   ├── rtl_view/               # yosys 生成的 RTL 图
│   └── synth_fa.ys             # yosys 综合脚本
└── comparator/
    ├── RTL/
    │   └── comparator.v
    ├── sim/
    │   └── tb_comparator.v
    ├── rtl_view/
    └── synth_cmp.ys
```

```
# 在 GitHub 新建空仓库 eie330-lab1 后：
mkdir eie330-lab1 && cd eie330-lab1
git init
mkdir -p full_adder/RTL full_adder/sim full_adder/rtl_view
mkdir -p comparator/RTL comparator/sim comparator/rtl_view
git add .
git commit -m "Lab1: project skeleton"
git branch -M main
git remote add origin <你的仓库地址>
git push -u origin main
```

> 关于 .vcd / .vvp 文件：仿真生成的波形和可执行文件属于临时产物，建议在仓库根目录放一个 .gitignore，写入 *.vcd、*.vvp、rtl_view/*.dot，只提交源代码与脚本；波形 CSV 和报告用图可保留。

第 3 章 六步开发流程（总览）

下图是每个任务都要完整走一遍的流程，也是你报告的叙事主线。记住红色回环箭头：任何一步验证不通过，都要回到对应步骤修正，再重新往下走。

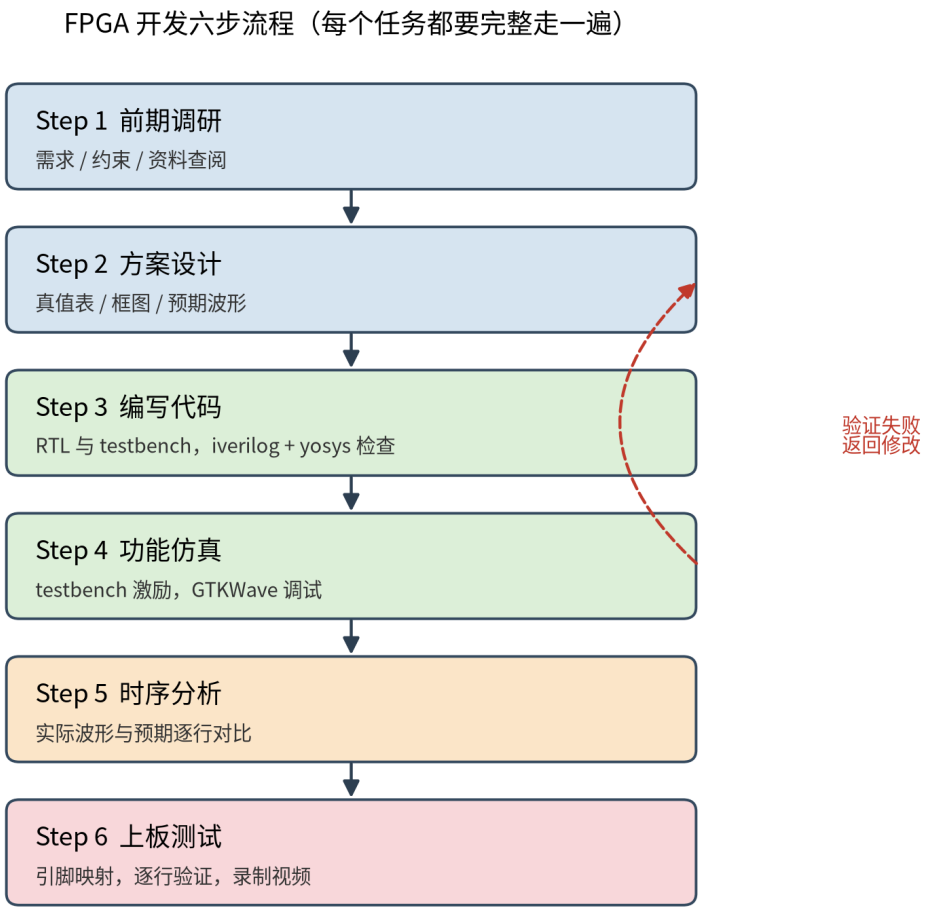


图 3-1 FPGA 开发六步流程（验证失败必须返回修正）

每个步骤的输入、动作与产出：

步骤	你要做的事	产出物
1 调研	列出功能需求；确认按键/LED 低有效；查 FPGA 数据手册资源数	可逐条检查的需求清单
2 设计	写真值表、端口表、结构框图、预期波形；确定设计决策并说明理由	设计规格 + 框图 + 预期波形
3 编码	写 RTL 与 testbench；iverilog 零警告编译；yosys 综合并对照框图	.v 代码、综合统计、RTL 视图
4 仿真	testbench 遍历全部输入组合；dump VCD；GTKWave 找第一个错误点	.vcd 波形、仿真打印结果
5 时序分析	波形与预期逐行对比；VCD→CSV→绘制正式波形图；说明零延迟局限	对比结论 + 报告波形图

步骤	你要做的事	产出物
6 上板	引脚映射；逐行验证真值表；录制视频；现象不符则返回 Step 3	引脚表、验证检查表、视频

> 工具能力边界（报告里要体现你理解）：功能仿真是零延迟理想模型，只能证明逻辑功能；yosys 做的是通用综合，不等于目标 FPGA
> 的真实布局布线；仿真器只是硬件模型。最终正确性必须由真实硬件测试确认。

第 4 章 任务 1-1：全加器（手把手）

4.1 Step 1 调研：需求与真值表

全加器实现 1 位二进制加法：A、B 是两个加数，Cin 是来自低位的进位，输出和 S 与向高位的进位 Cout。它与半加器的区别是多了进位输入 Cin，能级联成多位加法器。

完整真值表（共 8 行，必须全部验证）：

A	B	Cin	S（和）	Cout（进位）
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

需求清单（每条都要能被一个测试检查）：① 3 个按键分别对应 A、B、Cin；② 2 个 LED 分别对应 S、Cout；③ 按下为 1、亮灯为 1；④ 8 种组合全部符合真值表。

4.2 Step 2 设计：表达式、框图与预期波形

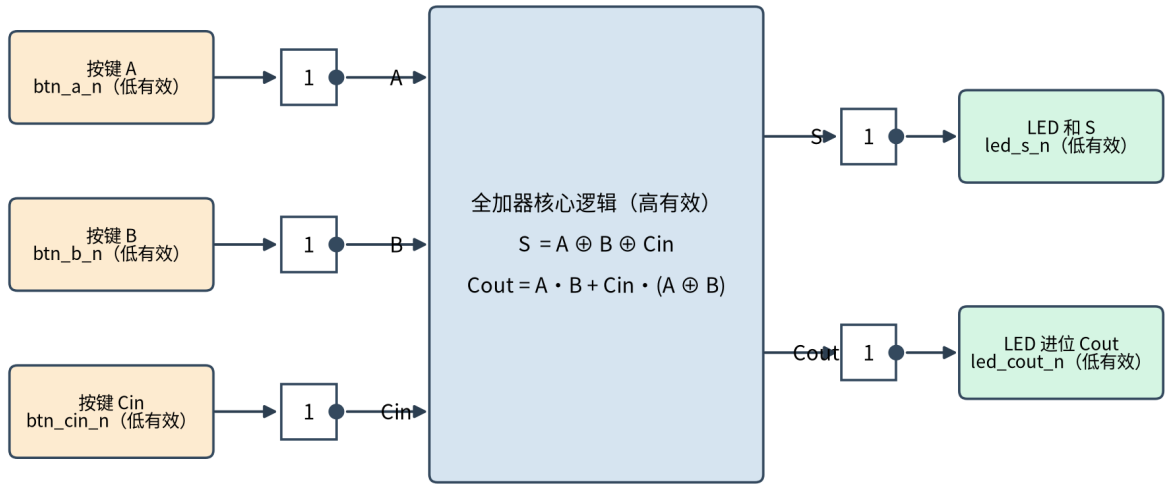
由真值表可得逻辑表达式：

$S = A \oplus B \oplus Cin$ $Cout = A{\cdot}B + Cin{\cdot}(A \oplus B)$

设计决策（报告中要写明理由）：

- 纯组合逻辑：输出只取决于当前输入，不需要时钟和触发器；
- 低有效处理放在端口：输入按键先取反得到高有效信号，核心逻辑全部按高有效设计，输出给 LED 前再取反。这样核心逻辑与教材一致、可读性最好；
- 数据流描述风格（assign）：电路是简单的门级组合关系，数据流描述最直接、可综合性好，也便于与表达式一一对照。

任务 1-1 全加器：系统结构框图（含低电平有效转换）



要点：板级信号低有效（_n 后缀），进入核心逻辑前先取反；核心逻辑全部按高有效设计；输出给 LED 前再取反。

图 4-1 全加器结构框图：板级信号低有效，核心逻辑高有效

写代码前先画出预期波形（每 10 ns 给一组输入，按 000→111 遍历）：

任务 1-1 全加器：预期输入 / 输出波形（每 10 ns 一组输入，共 8 组）

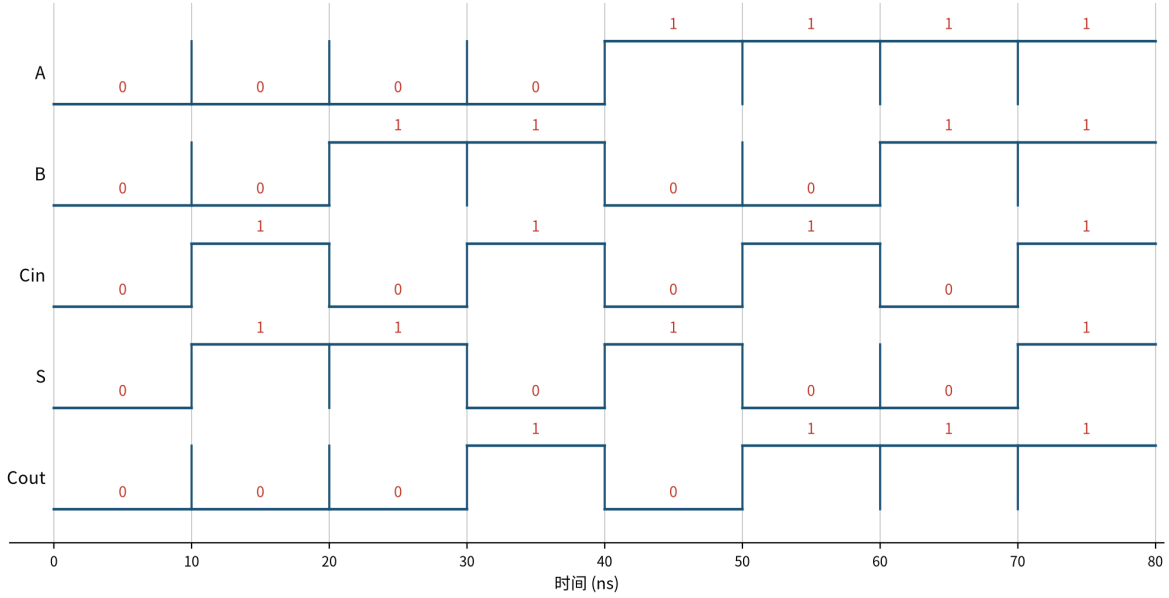


图 4-2 全加器预期输入/输出波形（Step 5 要用仿真结果与此图逐行对比）

4.3 Step 3 编写 RTL 设计代码

在 full_adder/RTL/full_adder.v 中写入以下代码（端口名带 _n 后缀表示低有效）：

```
//============================================================
// EIE330 Lab 1-1 : Full Adder
// File : RTL/full_adder.v
// Board buttons/LEDs are ACTIVE-LOW; the adder core is ACTIVE-HIGH.
// Visible behavior: button pressed = 1, LED on = 1.
//============================================================
module full_adder (
    input  wire btn_a_n,     // button for addend A       (press = 0)
```

```
    input  wire btn_b_n,     // button for augend B        (press = 0)
    input  wire btn_cin_n,   // button for carry-in Cin    (press = 0)
    output wire led_s_n,     // LED for sum S              (on = 0)
    output wire led_cout_n   // LED for carry-out Cout     (on = 0)
);

    // ---- input level conversion: active-low -> active-high ----
    wire A   = ~btn_a_n;
    wire B   = ~btn_b_n;
    wire Cin = ~btn_cin_n;

    // ---- active-high full adder core (pure combinational) ----
    wire S;
    wire Cout;
    assign S    = A ^ B ^ Cin;                    // sum
    assign Cout = (A & B) | (Cin & (A ^ B));      // carry-out

    // ---- output level conversion: active-high -> active-low ----
    assign led_s_n    = ~S;
    assign led_cout_n = ~Cout;

endmodule
```

代码 4-1 full_adder.v（GitHub 中提交完整文件；报告正文只放关键片段）

4.4 Step 3 语法检查与逻辑综合

进入任务目录，用 iverilog 做语法检查（-Wall 打开全部警告，-g2012 使用 SystemVerilog 风格语法）：

```
cd eie330-lab1/full_adder
iverilog -Wall -g2012 -o sim/full_adder_tb.vvp \
    RTL/full_adder.v sim/tb_full_adder.v
```

命令 4-1 iverilog 编译（testbench 将在 4.5 节创建；此处可先只编译 RTL 自查语法）

只检查 RTL 语法时可以运行：

```
iverilog -Wall -g2012 -o /tmp/fa_syn.v RTL/full_adder.v && echo SYNTAX_OK
```

接着编写 yosys 综合脚本 synth_fa.ys：

```
read_verilog RTL/full_adder.v
synth -top full_adder
stat
show -format png -prefix rtl_view/full_adder
```

代码 4-2 synth_fa.ys（stat 打印资源统计，show 生成 RTL 视图）

```
mkdir -p rtl_view
yosys -s synth_fa.ys
```

命令 4-2 运行综合；rtl_view/full_adder.png 即 RTL 视图

综合结果检查要点：

- RTL 视图的端口、门电路与图 4-1
  一致：应看到异或、与、或逻辑以及端口处的反相，没有多余逻辑；
- stat 统计中不应出现 latch（锁存器）或 $dff（触发器）——本设计是纯组合逻辑；
- iverilog 与 yosys 的任何 warning 都要查清并修复，不能忽略。

4.5 Step 4 编写 Testbench 并功能仿真

在 full_adder/sim/tb_full_adder.v 写入以下自检型测试平台：它遍历全部 8 种组合，内部带一个参考模型，自动比对实际输出与预期输出。注意按键是低有效，所以施加的板级电平是逻辑值的取反。

```
//============================================================
// EIE330 Lab 1-1 : Testbench for full_adder
// File : sim/tb_full_adder.v
// Exhaustive test: all 8 combinations of (A, B, Cin).
//============================================================
`timescale 1ns/1ps
module tb_full_adder;

    // device-under-test ports (board level, active-low)
    reg  btn_a_n;
    reg  btn_b_n;
    reg  btn_cin_n;
    wire led_s_n;
    wire led_cout_n;

    // active-high view for self-checking
    wire A    = ~btn_a_n;
    wire B    = ~btn_b_n;
    wire Cin  = ~btn_cin_n;
    wire S    = ~led_s_n;
    wire Cout = ~led_cout_n;

    // reference model (expected result)
    wire exp_s    = A ^ B ^ Cin;
    wire exp_cout = (A & B) | (Cin & (A ^ B));

    full_adder dut (
        .btn_a_n    (btn_a_n),
        .btn_b_n    (btn_b_n),
        .btn_cin_n  (btn_cin_n),
        .led_s_n    (led_s_n),
        .led_cout_n (led_cout_n)
    );

    integer errors = 0;

    // apply one board-level button vector, then check against model
    task apply_input(input [2:0] btn_vec);
        begin
            {btn_a_n, btn_b_n, btn_cin_n} = btn_vec;
            #1;                                  // wait for combinational settle
            if (S !== exp_s || Cout !== exp_cout) begin
                $display("FAIL A=%b B=%b Cin=%b : got S=%b Cout=%b, expect %b %b",
                         A, B, Cin, S, Cout, exp_s, exp_cout);
                errors = errors + 1;
            end else begin
                $display("OK   A=%b B=%b Cin=%b -> S=%b Cout=%b",
                         A, B, Cin, S, Cout);
            end
            #9;
        end
    endtask

    initial begin
        $dumpfile("full_adder.vcd");             // waveform file
        $dumpvars(0, tb_full_adder);
```

```
        //  A B Cin | expected S Cout ; button level = ~logic level
        apply_input(3'b111);   // 0 0 0 | 0 0
        apply_input(3'b110);   // 0 0 1 | 1 0
        apply_input(3'b101);   // 0 1 0 | 1 0
        apply_input(3'b100);   // 0 1 1 | 0 1
        apply_input(3'b011);   // 1 0 0 | 1 0
        apply_input(3'b010);   // 1 0 1 | 0 1
        apply_input(3'b001);   // 1 1 0 | 0 1
        apply_input(3'b000);   // 1 1 1 | 1 1

        if (errors == 0) $display("RESULT: ALL 8 CASES PASSED");
        else             $display("RESULT: %0d CASE(S) FAILED", errors);
        $finish;
    end

endmodule
```

代码 4-3 tb_full_adder.v（含 $dumpfile/$dumpvars 与自动比对）

编译、运行仿真、打开波形：

```
iverilog -Wall -g2012 -o sim/full_adder_tb.vvp \
    RTL/full_adder.v sim/tb_full_adder.v
vvp sim/full_adder_tb.vvp
gtkwave full_adder.vcd &
```

命令 4-3 编译 → 仿真 → 查看波形

仿真正确时，终端应逐行打印（与真值表完全一致）：

```
OK   A=0 B=0 Cin=0 -> S=0 Cout=0
OK   A=0 B=0 Cin=1 -> S=1 Cout=0
OK   A=0 B=1 Cin=0 -> S=1 Cout=0
OK   A=0 B=1 Cin=1 -> S=0 Cout=1
OK   A=1 B=0 Cin=0 -> S=1 Cout=0
OK   A=1 B=0 Cin=1 -> S=0 Cout=1
OK   A=1 B=1 Cin=0 -> S=0 Cout=1
OK   A=1 B=1 Cin=1 -> S=1 Cout=1
RESULT: ALL 8 CASES PASSED
```

输出 4-1 预期终端输出（出现 FAIL 说明设计或测试平台有 bug）

GTKWave 调试操作：① 左上 SST 窗口展开并选中 tb_full_adder；②
把要看的信号（A、B、Cin、S、Cout 及板级信号）选中后右键 Append/Insert 到 Signals 窗口；③
点工具栏 Zoom Fit（放大镜图标）显示全部时间；④
找到实际波形与预期第一次出现偏差的位置定位问题。

> 报告不能放 GTKWave 截图：GTKWave 截图只是调试辅助，不允许作为报告插图。报告波形必须按 4.6 节由 VCD 数据绘制。

4.6 Step 5 时序分析与波形绘图

第一步：逐行对比仿真波形与图 4-2 的预期波形，8 段必须完全一致；不一致时判断是设计、代码还是 testbench 问题并返回修正。

第二步：把 VCD 转成 CSV。把下面脚本保存为 vcd2csv.py（放在仓库根目录或各任务目录）：

```
#!/usr/bin/env python3
# vcd2csv.py - dump every signal value at every change time to CSV.
# Usage: python3 vcd2csv.py input.vcd output.csv
import sys
from vcdvcd import VCDVCD

vcd = VCDVCD(sys.argv[1])

sigs = []
for s in vcd.data.values():
    name = s.references[-1].replace(".", "_")
    sigs.append((name, [(int(t), v) for t, v in s.tv]))

times = sorted({t for _, tv in sigs for t, _ in tv})

def value_at(tv, t):
    val = "x"
    for ct, cv in tv:
        if ct <= t:
            val = cv
        else:
            break
    return val

with open(sys.argv[2], "w") as f:
    f.write("time," + ",".join(n for n, _ in sigs) + "\n")
    for t in times:
        f.write(str(t) + "," + ",".join(value_at(tv, t) for _, tv in sigs) + "\n")
print("wrote", sys.argv[2])
```

代码 4-4 vcd2csv.py（依赖 pip 包 vcdvcd）

第三步：把下面绘图脚本保存为 plot_wave.py，它读取 CSV 并画出可放入报告的数字波形：

```
#!/usr/bin/env python3
# plot_wave.py - draw a report-ready digital waveform from CSV.
# Usage: python3 plot_wave.py data.csv "Fig title" out.png [sig ...]
import sys
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv(sys.argv[1])
title, out = sys.argv[2], sys.argv[3]
cols = sys.argv[4:] if len(sys.argv) > 4 else \
       [c for c in df.columns if c != "time"]

fig, ax = plt.subplots(figsize=(10, 0.75 * len(cols) + 1.4))
for i, c in enumerate(cols):
    base = len(cols) - 1 - i
    ax.step(df["time"], base + df[c].astype(int), where="post", lw=1.7)
    ax.axhline(base, color="#DDDDDD", lw=0.7)
    ax.text(df["time"].iloc[0] - 1.5, base + 0.5, c,
            ha="right", va="center", fontsize=11)
ax.set_yticks([])
ax.set_xlabel("time (ns)")
ax.set_title(title)
plt.tight_layout()
plt.savefig(out, dpi=200)
print("wrote", out)
```

代码 4-5 plot_wave.py（由数据绘制波形，可直接作为报告图）

执行转换与绘图（信号名按 CSV 表头选择，通常选高有效信号 A B Cin S Cout）：

```
python3 vcd2csv.py full_adder.vcd full_adder.csv
python3 plot_wave.py full_adder.csv \
    "Full adder simulated waveform" fa_wave_sim.png \
    A B Cin S Cout
```

命令 4-4 VCD → CSV → 报告波形图

第三步（分析结论）：在报告中明确写出——功能仿真是零延迟模型，它验证了全部 8 种组合的逻辑功能，但不包含真实门延迟与线延迟；在真实传播延迟下能否正常工作，只能靠 Step 6 上板确认。

4.7 Step 6 上板测试

先做引脚映射（下表信号名按要求文档的 B2–B5、led1–led5 示例；具体物理引脚编号必须查你们开发板原理图/课程约束模板填写）：

模块端口	板上器件	板上信号名（示例）	有效电平
btn_a_n	按键（输入 A）	B2	低
btn_b_n	按键（输入 B）	B3	低
btn_cin_n	按键（输入 Cin）	B4	低
led_s_n	LED（输出 S）	led1	低
led_cout_n	LED（输出 Cout）	led2	低

约束文件格式取决于 FPGA 厂商，选择与你开发板对应的一种（详见第 6 章）。烧录后逐行按下按键、观察 LED，按下表逐项打勾，并录制验证视频：

按下的键（=1）	A B Cin	应亮 LED	实测是否一致
都不按	0 0 0	全灭	☐
Cin	0 0 1	S	☐
B	0 1 0	S	☐
B、Cin	0 1 1	Cout	☐
A	1 0 0	S	☐
A、Cin	1 0 1	Cout	☐
A、B	1 1 0	Cout	☐
A、B、Cin	1 1 1	S、Cout	☐

第 5 章 任务 1-2：一位数字比较器（手把手）

5.1 Step 1 调研：需求与真值表

一位比较器比较两个 1 位输入 A、B
的大小，输出三个互斥信号：A<B、A=B、A>B，任意时刻有且只有一个输出为 1。比较器是 ALU、排序电路、地址译码的基础部件。

A	B	lt（A<B）	eq（A=B）	gt（A>B）	亮灯
0	0	0	1	0	仅第 2 个
0	1	1	0	0	仅第 1 个
1	0	0	0	1	仅第 3 个
1	1	0	1	0	仅第 2 个

5.2 Step 2 设计

由真值表得表达式：

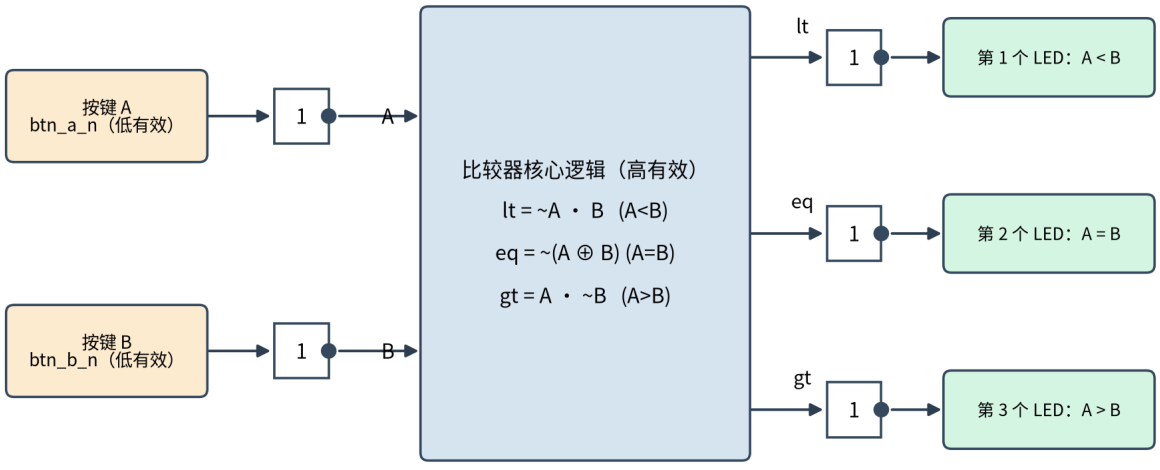
lt = ~A · B （A 小于 B）

eq = ~(A ⊕ B) （A 等于 B，即同或 A ⊙ B）

gt = A · ~B （A 大于 B）

- 同样是纯组合逻辑，无需时钟；
- 低有效处理方式与全加器一致：端口处取反，核心逻辑高有效；
- 采用数据流描述，三个 assign 与三个输出一一对应。

任务 1-2 一位比较器：系统结构框图（含低电平有效转换）



要点：任意输入下 lt / eq / gt 有且只有一个为 1，因此板上始终只有一个 LED 点亮。

图 5-1 比较器结构框图

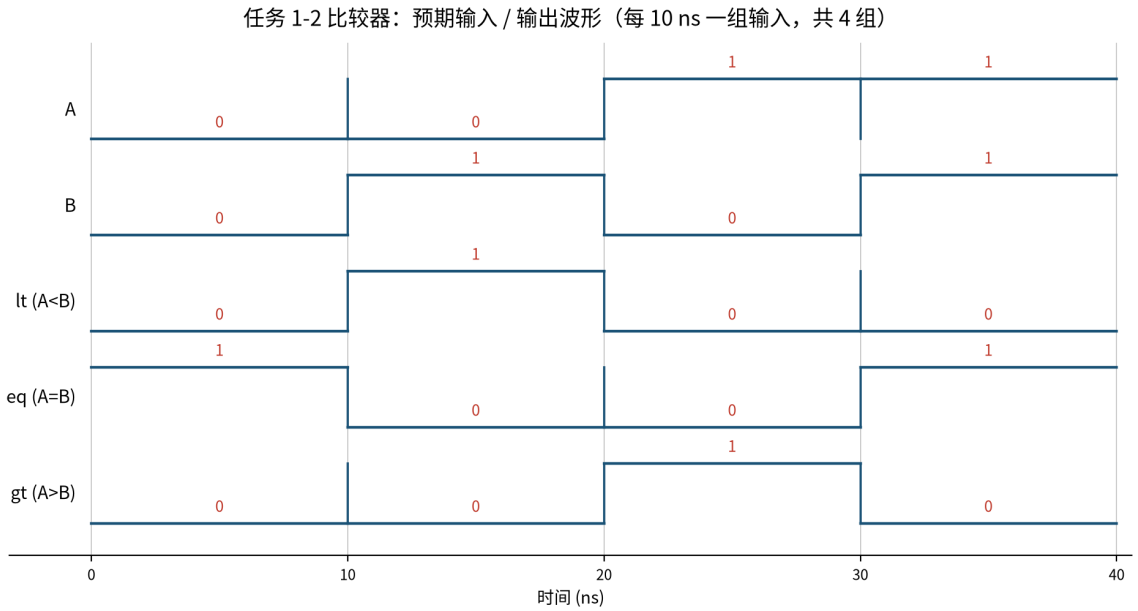


图 5-2 比较器预期输入/输出波形（4 组输入）

5.3 Step 3 RTL 代码与综合

```
//============================================================
// EIE330 Lab 1-2 : One-bit Digital Comparator
// File : RTL/comparator.v
// Buttons/LEDs active-low at board; core logic active-high.
//============================================================
module comparator (
    input  wire btn_a_n,
    input  wire btn_b_n,
    output wire led_lt_n,    // 1st LED: A < B
    output wire led_eq_n,    // 2nd LED: A = B
    output wire led_gt_n     // 3rd LED: A > B
);

    // input level conversion (active-low -> active-high)
    wire A = ~btn_a_n;
    wire B = ~btn_b_n;

    // active-high comparator core
    wire lt = (~A) &  B;      // A < B
    wire eq = ~(A ^ B);       // A = B (XNOR)
    wire gt =  A  & (~B);     // A > B

    // output level conversion (active-high -> active-low)
    assign led_lt_n = ~lt;
    assign led_eq_n = ~eq;
    assign led_gt_n = ~gt;

endmodule
```

代码 5-1 comparator.v

yosys 脚本 synth_cmp.ys 与综合命令：

```
read_verilog RTL/comparator.v
synth -top comparator
stat
show -format png -prefix rtl_view/comparator
```

代码 5-2 synth_cmp.ys

```
iverilog -Wall -g2012 -o /tmp/cmp_syn.v RTL/comparator.v && echo SYNTAX_OK
mkdir -p rtl_view
yosys -s synth_cmp.ys
```

命令 5-1 语法检查与综合（同样不应出现 latch / 触发器）

5.4 Step 4 Testbench 与仿真

```
//============================================================
// EIE330 Lab 1-2 : Testbench for comparator
// File : sim/tb_comparator.v
// Exhaustive test: all 4 combinations of (A, B).
//============================================================
`timescale 1ns/1ps
module tb_comparator;

    reg  btn_a_n;
    reg  btn_b_n;
    wire led_lt_n;
    wire led_eq_n;
    wire led_gt_n;

    // active-high view
    wire A  = ~btn_a_n;
    wire B  = ~btn_b_n;
    wire lt = ~led_lt_n;
    wire eq = ~led_eq_n;
    wire gt = ~led_gt_n;

    // reference model
    wire exp_lt = (~A) &  B;
    wire exp_eq = ~(A ^ B);
    wire exp_gt =  A  & (~B);

    comparator dut (
        .btn_a_n  (btn_a_n),
        .btn_b_n  (btn_b_n),
        .led_lt_n (led_lt_n),
        .led_eq_n (led_eq_n),
        .led_gt_n (led_gt_n)
    );

    integer errors = 0;

    task apply_input(input [1:0] btn_vec);
        begin
            {btn_a_n, btn_b_n} = btn_vec;
            #1;
            if (lt !== exp_lt || eq !== exp_eq || gt !== exp_gt) begin
                $display("FAIL A=%b B=%b : got lt/eq/gt=%b%b%b, expect %b%b%b",
                         A, B, lt, eq, gt, exp_lt, exp_eq, exp_gt);
                errors = errors + 1;
            end else begin
                $display("OK   A=%b B=%b -> lt/eq/gt = %b%b%b",
                         A, B, lt, eq, gt);
            end
            #9;
        end
    endtask
```

```
    initial begin
        $dumpfile("comparator.vcd");
        $dumpvars(0, tb_comparator);

        //  A B | expected lt eq gt ; button level = ~logic
        apply_input(2'b11);    // 0 0 | 0 1 0
        apply_input(2'b10);    // 0 1 | 1 0 0
        apply_input(2'b01);    // 1 0 | 0 0 1
        apply_input(2'b00);    // 1 1 | 0 1 0

        if (errors == 0) $display("RESULT: ALL 4 CASES PASSED");
        else             $display("RESULT: %0d CASE(S) FAILED", errors);
        $finish;
    end

endmodule
```

代码 5-3 tb_comparator.v

```
iverilog -Wall -g2012 -o sim/comparator_tb.vvp \
    RTL/comparator.v sim/tb_comparator.v
vvp sim/comparator_tb.vvp
gtkwave comparator.vcd &
```

命令 5-2 编译、仿真、查看波形

```
OK   A=0 B=0 -> lt/eq/gt = 010
OK   A=0 B=1 -> lt/eq/gt = 100
OK   A=1 B=0 -> lt/eq/gt = 001
OK   A=1 B=1 -> lt/eq/gt = 010
RESULT: ALL 4 CASES PASSED
```

输出 5-1 比较器预期终端输出

5.5 Step 5 时序分析与波形绘图

```
python3 vcd2csv.py comparator.vcd comparator.csv
python3 plot_wave.py comparator.csv \
    "Comparator simulated waveform" cmp_wave_sim.png \
    A B lt eq gt
```

命令 5-3 生成比较器报告波形（脚本同 4.6 节）

把绘制出的波形与图 5-2 逐行对比，4 段必须一致；并在报告中再次说明零延迟仿真的局限。

5.6 Step 6 上板测试

模块端口	板上器件	板上信号名（示例）	有效电平
btn_a_n	按键（输入 A）	B2	低
btn_b_n	按键（输入 B）	B3	低
led_lt_n	第 1 个 LED（A<B）	led1	低
led_eq_n	第 2 个 LED（A=B）	led2	低
led_gt_n	第 3 个 LED（A>B）	led3	低

按下的键（=1）	A B	应亮 LED	实测
都不按	0 0	仅第 2 个（=）	☐
B	0 1	仅第 1 个（<）	☐
A	1 0	仅第 3 个（>）	☐
A、B	1 1	仅第 2 个（=）	☐

第 6 章 开发板测试通用流程

6.1 引脚约束文件

综合通过后，要把模块端口绑定到开发板的物理引脚。约束文件格式由 FPGA 厂商决定，请使用课程开发板对应的格式与引脚编号。下面给出两种常见格式作为示例。

Lattice iCE40（yosys + nextpnr + Project IceStorm 流程，.pcf 文件）：

```
# full_adder.pcf  -- pin constraints (REPLACE pins with your board's!)
set_io btn_a_n    23
set_io btn_b_n    24
set_io btn_cin_n  25
set_io led_s_n    10
set_io led_cout_n 11
```

代码 6-1 .pcf 约束示例（引脚号仅为占位）

Xilinx（Vivado 流程，.xdc 文件）：

```
# full_adder.xdc -- REPLACE pins and IOSTANDARD with your board's!
set_property PACKAGE_PIN U16 [get_ports btn_a_n]
set_property IOSTANDARD LVCMOS33 [get_ports btn_a_n]
set_property PACKAGE_PIN E19 [get_ports {led_s_n[*]}]
```

代码 6-2 .xdc 约束示例（仅示格式）

> 不要凭记忆写引脚号：引脚编号、电平标准（IOSTANDARD）必须与开发板原理图和课程提供的约束模板一致；未使用的引脚保持不连接。错误的引脚约束是“仿真全对、上板全错”的最常见原因。

6.2 综合、布局布线与烧录

iCE40 开源工具链的典型命令序列（型号参数 hx1k、封装 tq144 等必须按实际板子替换，课程若提供了一键脚本/Makefile 请直接使用）：

```
yosys -p "synth_ice40 -top full_adder -json full_adder.json" \
    RTL/full_adder.v
nextpnr-ice40 --hx1k --package tq144 \
    --json full_adder.json --pcf full_adder.pcf --asc full_adder.asc
icepack full_adder.asc full_adder.bin
iceprog full_adder.bin
```

命令 6-1 iCE40 综合→布局布线→生成比特流→烧录（参数按板子替换）

Xilinx / Intel 板子则在 Vivado / Quartus 图形工程中添加 RTL 与约束文件，依次 Run Synthesis → Run Implementation → Generate Bitstream → Program Device。

6.3 逐行验证与视频录制

- 严格按第 4.7 / 5.6 节检查表逐行操作，每一行都要实际按到、看到并记录；
- 录制开发板窗口/实物的视频：视频中依次展示每一种输入组合与对应 LED，建议口头或字幕说明当前是哪一行；
- 视频上传到视频网站，把链接写入报告；可在报告中插入少量截图作为补充，但不能替代视频；
- 若板级现象与仿真不一致：先查引脚约束与低有效取反，再回 Step 3 修改代码，重新综合烧录。

现象全反了怎么办：不按按键灯却亮、按下灯灭——几乎都是漏做低有效转换。确认输入侧 A=~btn_a_n、输出侧 led_n=~S 两处取反都在。

第 7 章 实验报告怎么写

每个任务单独一份报告，结构严格按 a)–h) 组织；最后两份报告合并为一个 PDF，每份报告有独立封面。下面逐节说明写什么。

7.1 封面

封面包含：课程名、任务名（Lab 1-1 Full Adder / Lab 1-2 One-bit Comparator）、每位成员姓名与学号、日期；并在合适位置给出 GitHub 仓库链接与验证视频链接。

7.2 a) Introduction（引言）

- 背景：1-1 写清什么是全加器、它与半加器的区别（有无 Cin）、在多位加法器/ALU 中的用途；1-2 写清比较器的功能与应用；
- 实验目的：掌握 FPGA 开发流程与组合逻辑设计、仿真、上板方法；
- 方案与结果概述：一两段话说你用什么方案实现、验证了多少种组合、结果如何；
- 组员贡献：用列表写明每位成员姓名、学号、具体承担的工作。

7.3 b) Requirements analysis（需求分析）

- 实验目标与完整功能需求，附完整真值表（1-1 八行、1-2 四行）；
- 量化的设计约束：按键/LED 低电平有效的电气行为、不需要时钟、FPGA 提供的引脚数与逻辑单元数（查数据手册并引用）；
- 推导得到的设计规格：端口名称、方向、有效电平；预期波形；
- 体现可追溯性：让读者能从每条需求 → 规格项 → 设计元素 → 测试用例一路对应。

7.4 c) System design（系统设计）

- 结构框图（类似图 4-1 / 5-1），每个信号都要标注；说明用几个模块及理由（电路简单，单模块即可，但要说明为什么）；
- 预期输入/输出波形；
- 设计决策与理由：组合还是时序、低有效如何处理、为什么选数据流/行为/结构描述风格。

7.5 d) Implementation（实现）

- 用等宽字体展示关键代码片段说明设计如何翻译成代码；不要把全部代码贴进报告，完整代码放 GitHub 并给出链接；
- 给出 iverilog 语法检查、yosys 综合无警告通过的证据（终端输出截图或文字）；
- 插入 yosys RTL 视图并解读：是否与框图一致？stat 的资源数字说明什么？有没有意外的锁存器/触发器/冗余逻辑？

7.6 e) Testing and results（测试与结果）—— 权重最高

- 验证计划：测哪些用例、为什么这些用例足以证明正确（穷举全部组合）；
- testbench 工作原理：激励如何施加、预期输出是什么、如何自动比对；testbench 完整代码同样放 GitHub；

- 结果：每种情况的预期 vs 实际输出（用表格），必要时给出关键内部变量；解释任何差异；
- 波形图：由仿真数据绘制（VCD→CSV→绘图），每张图编号、加图题、正文引用；禁止 GTKWave 截图；
- 板级验证：逐项检查表（类似 4.7 / 5.6）、验证视频链接（必须）、截图（可选补充）。

7.7 f) Discussion（讨论，建议写）

- 诚实说明各工具局限：仿真零延迟、yosys 是通用综合而非真实 FPGA 实现、仿真器≠真实板子；
- 遇到的 bug：如何发现、波形/RTL 视图显示了什么、如何解决（这是加分项）；
- 限定时间内未解决的问题与可能的解决思路；其他有趣观察。

7.8 g) Conclusion（结论）

简要总结设计方案与全部验证结果（8 组 / 4
组用例是否全部通过、上板是否全部符合），并总结讨论中的核心观点。

7.9 h) References（参考文献）

所有引用资料用 IEEE 引用格式列出，例如：

```
[1] S. Brown and Z. Vranesic, Fundamentals of Digital Logic with
     Verilog Design, 3rd ed. New York: McGraw-Hill, 2017.
[2] Claire Wolf, "Yosys Open SYnthesis Suite," YosysHQ documentation.
     [Online]. Available: https://yosyshq.net/yosys/
[3] S. Williams, "Icarus Verilog," documentation. [Online]. Available:
     https://steveicarus.github.io/iverilog/
[4] "GTKWave," documentation. [Online]. Available:
     https://gtkwave.sourceforge.net/
[5] <FPGA Vendor>, "<Your FPGA> Datasheet," rev. X, Year.  // 填实际芯片手册
```

代码 7-1 IEEE 格式参考文献示例（第 [5] 条替换为你实际查阅的芯片数据手册）

7.10 图表与代码规范（硬性要求）

- 每张图、每张表都编号、有标题、并在正文中被引用（如“见图 4-1”）；
- 波形必须由数据绘制；GTKWave 截图只能用于调试；
- 板级验证相反：视频链接为必须，截图仅为可选补充；
- 一两行短代码可以文字形式出现，长代码一律给 GitHub 链接，不允许代码截图；
- 如使用 AI 辅助，按课程 AI 政策在报告中声明哪些部分由 AI 协助完成。

第 8 章 提交前总检查清单

提交前逐项打勾，任何一项不确定都不要提交。

8.1 流程与代码检查

☐	两个任务的 RTL/ 与 sim/ 目录齐全，命名、缩进、注释规范
☐	代码只使用可综合语句，无锁存器、无多重驱动、无未声明信号
☐	按键输入与 LED 输出的低有效取反都已实现
☐	iverilog -Wall 编译零错误零警告
☐	yosys 综合无警告，RTL 视图与自绘框图一致，无 latch/$dff
☐	testbench 含 $dumpfile/$dumpvars，全加器 8 组、比较器 4 组全部 PASSED
☐	仿真波形与预期波形逐行对比一致
☐	报告波形由 VCD→CSV→脚本绘制，不是 GTKWave 截图
☐	上板逐行验证全部通过，并录制了视频

8.2 报告检查

☐	每个任务一份报告，各有独立封面（任务名、姓名、学号）
☐	a)–h) 各节齐全，顺序正确
☐	完整真值表、端口表、框图、预期/实测波形齐全
☐	所有图表均编号、有标题、正文有引用
☐	报告只含关键代码片段，完整代码在 GitHub 且链接可访问
☐	包含 iverilog/yosys 无警告证据与 RTL 视图解读
☐	包含预期 vs 实际结果表、板级检查表、验证视频链接
☐	讨论了工具局限（零延迟仿真、通用综合、仿真器≠硬件）
☐	参考文献为 IEEE 格式且覆盖实际引用的资料（含 FPGA 数据手册）
☐	写明了组员贡献；如使用 AI，已按政策声明

8.3 最终提交

☐	两份报告已合并为一个 PDF
☐	GitHub 仓库已公开/对教师可见，代码已 push
☐	验证视频已上传且链接在报告中可正常打开
☐	全文无抄袭内容（代码与文字）

附录 A 命令速查表

目的	命令
安装工具（Ubuntu）	sudo apt install iverilog gtkwave yosys graphviz python3-pip
安装 Python 包	pip3 install vcdvcd pandas matplotlib
语法检查	iverilog -Wall -g2012 -o /tmp/x.v RTL/xxx.v && echo OK
编译+testbench	iverilog -Wall -g2012 -o sim/x.vvp RTL/x.v sim/tb_x.v
运行仿真	vvp sim/x.vvp
查看波形	gtkwave xxx.vcd &
yosys 综合（脚本）	yosys -s synth_x.ys
yosys 一行命令	yosys -p "read_verilog RTL/x.v; synth -top x; stat"
VCD 转 CSV	python3 vcd2csv.py xxx.vcd xxx.csv
绘制波形图	python3 plot_wave.py xxx.csv "title" out.png A B ...
iCE40 比特流	yosys→nextpnr-ice40→icepack→iceprog（参数按板子）

附录 B 常见报错与排查

现象	原因与解决
iverilog: syntax error	检查 begin/end 是否配对、行末分号、`timescale、模块名拼写
iverilog: Unable to bind wire	实例化时端口名与模块定义不一致；核对 .端口名(信号)
warning: implicit declaration	信号未显式声明；补充 wire/reg 声明
yosys 报 latch inferred	if/case 分支不完整；组合逻辑改用 assign，或补 else/default
综合出现 $dff 触发器	不该有时钟的纯组合逻辑误用了 always@(posedge clk)；改为 @(*) 或 assign
没有生成 .vcd	忘记 $dumpvars，或没有运行 vvp；确认在运行目录下找文件
GTKWave 里没有信号	在 SST 窗口展开 testbench/DUT 层级，选中信号后 Append
仿真全对、上板全错	多为引脚约束错误或低有效未处理；查约束表与端口取反
不按键灯就亮/按下灯灭	低有效取反方向错误；输入 A=~btn_n、输出 led_n=~S
multiple drivers 报错	同一信号被多个 assign/always 驱动；合并为单一驱动
波形与预期不符	先查 testbench 激励（板级电平=逻辑值取反），再查 RTL 表达式

附录 C 报告封面模板（文字）

```
=========================================================
        EIE330 FPGA-based System Design
```

```
            Lab 1-X : <Full Adder / One-bit Comparator>

        Name (Student ID) :
        Group members     :
        Date              :

        GitHub repository : https://github.com/<user>/eie330-lab1
        Board test video  : <video URL>
=======================================================
```

代码 C-1 封面信息模板（两个任务各填一份）